

CODOC: Bidirectional Synchronization of a Human-Intent Feature Tree with Code

(method section draft)

Anonymous

1 The CODOC System

CODOC maintains a human-authored *feature tree* — a hierarchy of named units of intent — bidirectionally synchronized with the code of a repository. Unlike repository-level code graphs built *from* code as retrieval context for agents [4, 1, 2, 3], the tree is first-class authored intent: it is never regenerated from code, and the attribution of code to features is a secondary, continuously repaired index. Synchronization is asymmetric — code changes are *reflected* into the tree (Loop A, §1.2), while tree edits are *realized* into code by a coding agent (Loop B, §1.3) — and conflict resolution is deterministic: pending document intent always wins over code-side observations (§1.4).

Problem statement. *Given a repository whose indexed chunk set is $\mathcal{C}$ and a human-authored feature tree $\mathcal{T} = (\mathcal{F}, \prec)$, maintain (i) a binding map $\mathcal{B}: \mathcal{C} \rightarrow \mathcal{F} \cup \{\perp\}$ attributing every chunk to at most one feature, and (ii) the invariant that $\mathcal{T}$ expresses current human intent over the code, under concurrent edits to either side by humans and autonomous agents. Every mutation must be attributable (who, with what commitment, caused by what), and divergence must resolve to a synchronization state $\sigma = \text{in_sync}$ without losing authored intent.*

1.1 Preliminaries

Definition 1 (Chunk set). The repository is indexed into a set of AST-level chunks $\mathcal{C} = \{c_1, \dots, c_n\}$, where each chunk $c = (\text{file}, \text{sym}, h^{\text{tok}}, h^{\text{ast}})$ carries its file path, its qualified symbol path within the file, a *content hash* h^{tok} over the comment- and whitespace-normalized token stream, and an *AST-shape hash* h^{ast} over the token-erased tree structure. The pair $(\text{file}, \text{sym})$ is the chunk’s identity key; the two hashes are its relocation signals (Definition 6).

Definition 2 (Feature tree). The feature tree is $\mathcal{T} = (\mathcal{F}, \prec)$, where $\mathcal{F}$ is a finite set of features $f = (\text{id}, \text{title}, \text{desc}, \text{realized})$ and $\prec \subseteq \mathcal{F} \times \mathcal{F}$ is a forest-shaped parent relation. Titles and descriptions are free prose authored by humans (or proposed by agents and accepted by humans); *realized* $\in \{0, 1\}$ marks whether the feature is backed by code yet (*realized* = 0 denotes an accepted plan placeholder, flipped to 1 by the first binding).

Definition 3 (Binding map). Attribution is a partial function $\mathcal{B}: \mathcal{C} \rightarrow \mathcal{F} \cup \{\perp\}$: each chunk binds to at most one feature (enforced as a uniqueness constraint on the chunk identity key), while one feature typically owns many chunks across many files. Each binding stores the bound chunk’s h^{tok} as its *fingerprint*, making staleness locally detectable. We write $\mathcal{B}^{-1}(f) \subseteq \mathcal{C}$ for the chunks a feature owns and define coverage $\text{cov}(\mathcal{B}) = |\{c : \mathcal{B}(c) \neq \perp\}|/|\mathcal{C}|$.

Definition 4 (Code graph). Following the heterogeneous-graph convention of repository-level code representations [1, 2], the dependency structure of the code is $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ with node-type map $\tau: \mathcal{V} \rightarrow \mathcal{A}$, $\mathcal{A} = \{\text{file}, \text{class}, \text{function}\}$, and edge-type map $\varphi: \mathcal{E} \rightarrow \mathcal{R}$, $\mathcal{R} = \{\text{contain}, \text{call}, \text{import}, \text{inherit}\}$. $\mathcal{G}$ is a derived cache, rebuildable from $\mathcal{C}$ at any time. The binding map lifts $\mathcal{G}$ to a feature-coupling graph and defines the placement heuristic

$$\text{nbf}(c) = \arg \max_{f \in \mathcal{F}} |\{(v, v') \in \mathcal{E} : v' = \text{sym}(c), \mathcal{B}(\text{chunk}(v)) = f\}|, \quad (1)$$

the feature owning the most call/import edges incident to a new chunk c — the deterministic fallback of Loop A’s coverage net (Algorithm 1, lines 20–24).

Definition 5 (Changeset). A pass over the index yields $\Delta = (\Delta^+, \Delta^-, \Delta^\sim)$: chunks *added* (present, unbound), *removed* (a binding whose key has left the index), and *modified* (bound, but $h^{\text{tok}} \neq \text{fingerprint}$). CODOC computes Δ in two modes: a *temporal* diff of consecutive index snapshots (cheap, runs on every save) and a *state-based* reconciliation that compares the index directly against $\mathcal{B}$ (idempotent and self-healing; the authority pass). Only the latter may retire features or amend prose, so destructive judgments are never made from a transient mid-edit view.

Definition 6 (Relocation). For $(r, a) \in \Delta^- \times \Delta^+$: a is a *move* of r iff $h_a^{\text{tok}} = h_r^{\text{tok}}$ (identical content at a new key), and a *rename* of r iff $h_a^{\text{ast}} = h_r^{\text{ast}}$ uniquely within the same file (same shape, new name; 1:1 matches only). Relocations re-attach attribution deterministically — no model call, no risk of dropped or duplicated attribution.

Definition 7 (Change ledger). Every mutation of $(\mathcal{T}, \mathcal{B})$ is an event in an append-only ledger L . A ledger entry $\ell = (op, f, \alpha, \mu, \delta)$ records the operation, its target, the *actor* $\alpha \in \{\text{human}\} \cup \text{Agents} \cup \{\text{loop}\}$, the *mode* $\mu \in \{\text{pen}, \text{suggest}, \text{auto}\}$ (committed edit, reviewable proposal, or mechanical maintenance), and a *causality link* δ — the directive, suggestion, or event id this change implements ($\perp$ if spontaneous). A *proposal* is an unapplied event; accepting it applies the operation and consumes the event.

Definition 8 (Hold set and synchronization state). $H \subseteq \mathcal{F}$ is the set of features with pending document-ahead intent: a live suggestion not yet drained, or a queued directive not yet implemented. The synchronization state is $\sigma \in \{\text{in_sync}, \text{code_drift}, \text{tree_dirty}, \text{awaiting_impl}, \text{realizing}\}$, derived (not stored): pending code-side proposals put $\sigma = \text{code_drift}$, a non-empty directive queue forces $\sigma = \text{awaiting_impl}$ (a later code-side pass cannot silently clobber queued work), and an active implementation pass reports $\sigma = \text{realizing}$.

Table 1 summarizes the notation.

1.2 Loop A: Reflecting Code into the Tree

Loop A (Algorithm 1) closes the gap from code to intent. Its design principle is *determinism first*: every change that can be resolved mechanically — refreshing fingerprints, detaching deleted chunks, carrying attribution across moves and renames — is resolved without a model (lines 4–8). A single LLM pass (line 12) handles only the judgment calls that remain: placing genuinely new code, repairing features that lost their last binding, and (in the authority pass) refreshing prose whose code changed underneath it. The pass sees the residual changeset, a file-locality subtree of $\mathcal{T}$, and *every* feature title; supplying the full title set to one call is what prevents duplicate features without any post-hoc deduplication machinery.

Model output is not trusted with destructive authority. Proposed operations are split by safety: binding maintenance and small description amendments (bounded by an edit-distance ratio) auto-apply with $\mu = \text{auto}$; structural operations (add / move / retire) become unapplied proposal events with $\mu = \text{suggest}$

Table 1: Notation.

Symbol	Meaning
$\mathcal{C}$	chunk set; $c = (file, sym, h^{\text{tok}}, h^{\text{ast}})$
$\mathcal{T} = (\mathcal{F}, \prec)$	feature tree (features + forest parent relation)
$\mathcal{B}: \mathcal{C} \rightarrow \mathcal{F} \cup \{\perp\}$	binding map (each chunk owned by ≤ 1 feature)
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$	heterogeneous code dependency graph (derived cache)
$\Delta = (\Delta^+, \Delta^-, \Delta^\sim)$	changeset: added / removed / modified chunks
$h^{\text{tok}}, h^{\text{ast}}$	content hash (move-invariant), AST-shape hash (rename-invariant)
$L; \ell = (op, f, \alpha, \mu, \delta)$	append-only ledger; entry = op, target, actor, mode, cause
$H \subseteq \mathcal{F}$	hold set (features with pending doc-ahead intent)
σ	synchronization state
$\text{nbf}(c)$	graph-neighbor feature of a chunk (Eq. 1)
$D; d$	directive queue; a single realization directive

that a human accepts or rejects. Operations targeting held features are dropped (line 13; §1.4), and a *coverage net* (lines 20–24) guarantees no added chunk is silently dropped: anything the model failed to place attaches to its graph-neighbor feature $\text{nbf}(c)$ or surfaces as an explicit **add** proposal. Finally, stale-proposal garbage collection keeps the pass idempotent: a pending **add** whose chunks have since bound elsewhere, or a pending **retire** whose feature regained code, is dropped, so repeated passes converge to $\sigma = \text{in_sync}$ instead of accumulating false positives.

Two authority levels share this pipeline. The *temporal* pass (run on every save) computes Δ from consecutive snapshots and may never retire a feature or amend prose — a feature that looks emptied mid-edit usually rebinds on the next save. The *authority* pass (run at agent-session close, on daemon startup, and from explicit sync) computes Δ state-based against $\mathcal{B}$, is idempotent, recovers missed cycles, and is the only pass allowed to raise **retire** and stale-prose **amend** proposals.

1.3 Loop B: Realizing Tree Edits into Code

Loop B (Algorithm 2) closes the opposite gap. Edits to the rendered tree document are parsed and diffed against the store; verdicts on pending proposals are drained from the review inbox; and live document-ahead suggestions are applied by the loop itself (the human’s only verb on their own suggestion is *withdraw* — application is the agent side’s acknowledgment, stamped $\mu = \text{suggest}$ with $\delta =$ the suggestion id).

Two ordering invariants keep text and store coherent (lines 1 and 23). The text diff is snapshotted *before* any store mutation: verdict accepts move the store ahead of the on-disk text, and diffing afterwards would misread the stale text as a human edit that reverts the accepted change. Symmetrically, any pass that mutated the store re-renders the text before returning, so the next pass diffs a caught-up document.

Not every tree edit means “write code.” The *imperative gate* (line 15) classifies each applied edit: descriptive prose (“validates the token”) merely persists, while imperative prose (“should validate the token”, “Add retry logic”), an accepted unrealized plan node, or retiring a feature that owns code each mint a *directive* d — a natural-language work order assembled from the feature’s description and its bound symbols, scoped to the files the feature owns. Directives are queued for the user’s *live* coding-agent session rather than a headless model: the session implements each directive, reflects the new code back through the agent-side reflection API with $\delta = d$, and deletes the queue. Loop A then closes the cycle, with every resulting ledger entry carrying the causal chain from the original document edit.

Verdict semantics are deliberately asymmetric for the destructive case (line 6): accepting a machine-raised **retire** from the inbox is *detach-only* — the feature is hidden and its chunks freed for re-attribution, but no code-deletion directive is ever queued, because a retire raised from transient drift may be a false

Algorithm 1 Loop A — code $\rightarrow$ tree reflection (one pass)

Require: index snapshot of $\mathcal{C}$; store $(\mathcal{T}, \mathcal{B}, L)$; hold set H ; authority flag $auth$

Ensure: updated $(\mathcal{T}, \mathcal{B}, L)$; refreshed σ

```
1:  $\Delta \leftarrow \text{CHANGESSET}(\mathcal{C}, \mathcal{B})$   $\triangleright$  temporal diff, or state-based iff  $auth$ 
2:  $\text{GCSTALEPROPOSALS}(L, \Delta)$   $\triangleright$  drop superseded add/retire proposals
3: for  $c \in \Delta^\sim$  do  $\text{APPLY}(\text{refresh}(c), \mu=\text{auto})$ 
4: end for
5: for  $c \in \Delta^-$  do  $\text{APPLY}(\text{detach}(c), \mu=\text{auto})$ 
6: end for
7: for  $(r, a) \in \text{RELOCATIONS}(\Delta^-, \Delta^+)$  do  $\triangleright$  Def. 6:  $h^{\text{tok}}$  move /  $h^{\text{ast}}$  rename
8:    $\text{APPLY}(\text{attach}(a, \mathcal{B}(r)), \mu=\text{auto})$ 
9: end for
10:  $U \leftarrow \{c \in \Delta^+ : \mathcal{B}(c) = \perp\}$ ;  $E \leftarrow \{f : \mathcal{B}^{-1}(f) = \emptyset \wedge f \notin H\}$   $\triangleright$  unbound adds; emptied features
11: if  $U \neq \emptyset \vee E \neq \emptyset \vee (auth \wedge \text{STALEPROSE}(\Delta^\sim, H))$  then
12:    $ops \leftarrow \text{LLM}(\Delta, \text{SUBTREE}(\mathcal{T}, \Delta), \text{TITLES}(\mathcal{T}), \mathcal{G})$   $\triangleright$  one call per pass
13:    $ops \leftarrow \{o \in ops : \neg \text{SUPPRESSED BYHOLD}(o, H)\}$ 
14:   for  $o \in ops$  do
15:     if  $\text{SAFE}(o)$  then  $\text{APPLY}(o, \mu=\text{auto})$ 
16:     else  $\text{PROPOSE}(o, \mu=\text{suggest})$   $\triangleright$  unapplied event; human verdict pending
17:     end if
18:   end for
19: end if
20: for  $c \in U$  with  $\mathcal{B}(c) = \perp$  still do  $\triangleright$  coverage net: nothing is dropped
21:   if  $\text{nbfc}(c) \neq \perp$  then  $\text{APPLY}(\text{attach}(c, \text{nbfc}(c)), \mu=\text{auto})$ 
22:   else  $\text{PROPOSE}(\text{add}(c), \mu=\text{suggest})$ 
23:   end if
24: end for
25: return  $(\mathcal{T}, \mathcal{B}, L), \text{REFRESHSTATE}()$ 
```

positive and deleting code on a single click is the most destructive failure mode. Only an explicit human retire edit in the document text (or an agent proposal explicitly flagged to delete code) queues removal work.

1.4 Conflict Resolution: the Ledger and Doc-Wins Holds

Both loops route every decision through one classification table (Table 2): each detected change, from either side and by any actor, maps to exactly one reaction. Three properties follow.

Provenance. Every mutation is a ledger entry (Definition 7). The actor/mode pair separates *who* changed something from *how committed* the change is, which drives both review routing (only $\mu = \text{suggest}$ items need verdicts) and the interface’s authorship rendering; δ threads causality across the realize cycle, so a code change surfaced back by Loop A is grouped under the document edit that caused it.

Doc-wins holds. Conflict resolution is a deterministic priority rule rather than a merge heuristic. While $f \in H$ (a suggestion or queued directive is pending on f), code-side *intent* operations on f — amend, retire, move — are suppressed:

$$\text{suppress}(o, H) \iff o.\text{kind} \in \{\text{amend}, \text{retire}, \text{move}\} \wedge o.f \in H. \quad (2)$$

Table 2: The change-classification table. Every change, from either side and by any actor, maps to one reaction. **auto** reactions apply immediately; **suggest** reactions await a human verdict; **D** marks reactions that mint a realization directive.

#	Side	Detected change	Origin	Reaction
1	code	modified bound chunk	any	refresh binding (auto)
2	code	removed bound chunk	any	detach (auto); emptied $\wedge f \notin H \Rightarrow$ retire (suggest)
3	code	relocated chunk (Def. 6)	any	deterministic attach to prior feature (auto)
4	code	added unbound chunk	any	LLM placement (auto/suggest); fallback nbfd / add (auto)
5	code	in-place modify, realized + prose	any	stale-prose amend (small $\rightarrow$ auto, large $\rightarrow$ suggest)
6	code	any change during realize epoch	agent	rows 1–5 with $\delta =$ directive id
7	doc	descriptive amend / title edit	human, pen	apply immediately; no directive
8	doc	imperative amend / plan add / retire-with-code	human, pen	apply + mint directive (D)
9	doc	any edit	human, suggest	pending intent (hold); applied by Loop B drain $\rightarrow$ row 10
10	doc	structural op via reflection API	agent	pending proposal (suggest)
11	doc	safe op via reflection API	agent	auto-apply; ledger $\alpha =$ agent, $\mu =$ auto
12	verdict	accept / reject	human	apply+consume / delete (retire accept is detach-only)
13	code	drift on held feature ($f \in H$)	any	bindings maintained; amend/retire/move suppressed

Binding maintenance (attach/detach/refresh) is never suppressed — bindings are attribution, not intent, and must stay correct regardless of who is mid-edit. Holds release when the suggestion is drained or the directive queue completes, with a staleness backstop so an abandoned suggestion cannot hold a feature indefinitely. The rule is the tree-is-authoritative analogue of last-writer-wins: the side that expresses *intent* (the document) always outranks the side that merely *observes* (the code reflector).

Verdict grammar. Who resolves what is structural: accept/reject is exclusively a *human* verb on *code-ahead* items (machine proposals), while a *doc-ahead* suggestion is applied by the machine side and offers its human author only *withdraw*. No state is double-resolvable, which eliminates accept/revert races by construction.

1.5 Bootstrap

The initial tree is built by two scoped passes rather than one global clustering. A *per-file* pass issues one model call per source file, seeing only that file’s chunks and their incident graph edges — making cross-file “junk drawer” features structurally impossible — and a deterministic coverage step folds any chunk the model missed into the file’s largest feature. An *organization* pass then groups the resulting file-level features under a small number of thematic parents, using feature-to-feature coupling aggregated from $\mathcal{G}$ as evidence. Both passes emit ordinary ledger events through the same application seam as the loops, so bootstrap output is curated (and corrected) with exactly the same verdict machinery as any later change.

References

- [1] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL <https://arxiv.org/abs/2503.09089>.

- [2] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. Graphcoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024. URL <https://arxiv.org/abs/2406.07003>.
- [3] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. Codexgraph: Bridging large language models and code repositories via code graph databases. *arXiv preprint arXiv:2408.03910*, 2024. URL <https://arxiv.org/abs/2408.03910>.
- [4] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. Repograph: Enhancing AI software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=dw9VUsSHGB>.

Algorithm 2 Loop B — tree $\rightarrow$ code realization (one pass)

Require: tree text T_{txt} ; inbox verdicts V ; intents I ; store $(\mathcal{T}, \mathcal{B}, L)$ **Ensure:** updated $(\mathcal{T}, \mathcal{B}, L)$; directive queue D ; refreshed σ

```
1:  $ops_{\text{user}} \leftarrow \text{DIFF}(T_{\text{txt}}, \mathcal{T})$  ▷ snapshot BEFORE mutations (invariant 1)
2:  $D \leftarrow \emptyset$ 
3: for  $(e, \text{accept}) \in V$  do ▷ drain human verdicts on pending proposals
4:   if  $\text{accept}$  then
5:      $\text{APPLY}(e.op, \alpha=\text{human}, \mu=\text{pen}); \text{CONSUME}(e)$ 
6:     if  $e.op$  is retire  $\wedge \neg e.op.deleteCode$  then
7:        $\text{DETACHALL}(\mathcal{B}^{-1}(e.op.f))$  ▷ detach-only: never queue code deletion
8:     else if  $\text{IMPLIESCODE}(e.op)$  then  $D \leftarrow D \cup \{\text{DIRECTIVE}(e.op, \delta=e)\}$ 
9:     end if
10:   else  $\text{CONSUME}(e)$  ▷ reject: delete the proposal
11:   end if
12: end for
13: for  $o \in ops_{\text{user}}$  do ▷ direct edits are intentional: apply immediately
14:    $\text{APPLY}(o, \alpha, \mu \text{ from annotation channel (default human/pen)})$ 
15:   if  $\text{IMPLIESCODE}(o)$  then  $D \leftarrow D \cup \{\text{DIRECTIVE}(o, \delta=\ell_o)\}$ 
16:   end if
17: end for
18: for  $i \in I$  with payload, fresh, unsatisfied do ▷ doc-ahead suggestions: the loop applies them
19:    $\text{APPLY}(\text{amend}(i), \alpha=i.\alpha, \mu=\text{suggest}, \delta=i.id)$ 
20:   if  $\text{IMPLIESCODE}(i)$  then  $D \leftarrow D \cup \{\text{DIRECTIVE}(i, \delta=i.id)\}$ 
21:   end if
22: end for
23: if store mutated then  $\text{RENDER}(\mathcal{T}) \rightarrow T_{\text{txt}}$  ▷ invariant 2: text catches up
24: end if
25: if  $D \neq \emptyset$  then
26:   mint ids for  $D$ ; queue  $D$  for the live agent session;  $H \leftarrow H \cup \{d.f : d \in D\}$ 
27:    $\sigma \leftarrow \text{awaiting\_impl}$  ▷ the agent implements, reflects with  $\delta = d$ , clears the queue
28: else
29:    $\sigma \leftarrow \text{REFRESHSTATE}()$ 
30: end if
31: return  $(\mathcal{T}, \mathcal{B}, L), D, \sigma$ 
```
